

PreFlight Founder Playbook

1. The 30-Second Elevator Pitch

PreFlight is a local-first safety gate for AI-generated code.

When developers use AI coding tools, the AI can move incredibly fast. But it can also quietly change security rules, expose secret keys, break database permissions, or create production bugs that are hard to notice until it is too late.

PreFlight watches the code before it gets committed and asks one simple question:

Did the AI just introduce something dangerous?

It does this in two stages:

- ? First, it runs a fast offline structural scan on the developer's own machine.
- ? Then, for deeper architectural problems, it can route the risky finding into an MCP-powered patch flow that helps generate a safe fix.

The simplest way to pitch it:

PreFlight is the safety inspector for vibe-coded apps. It catches AI coding drift before it becomes a production incident.

Another version for judges:

Developers are now shipping code written by AI agents, but most teams do not have a guardrail between "the AI wrote it" and "it went to production." PreFlight is that guardrail. It scans code locally, detects structural security risks, scores them, and gives developers a clear release gate before dangerous changes land.

2. The Core Engine Explained

The Problem: AI Coding Drift

AI coding drift is what happens when an AI tool changes more than the developer intended.

A founder-friendly analogy:

Imagine asking a contractor to fix one window, but while you are not looking, they also move the door, remove a lock, and change the wiring. The house still looks fine from the outside, but it is now less safe.

In software, that looks like:

- ? A secret API key accidentally appearing in frontend code.
- ? A Supabase service role key being used in a place users can reach.
- ? A database table being created without Row-Level Security.
- ? A payment webhook being changed without idempotency protection.
- ? A Next.js middleware file letting requests through without an auth guard.
- ? A raw SQL query being built with string concatenation.

This is dangerous because the app can still "work" while the security model is broken.

The buzzword to use:

PreFlight detects AI Coding Drift: structural security and architecture changes silently introduced by AI coding agents.

Phase 1: Local AST Sweep

PreFlight's first phase is a local AST sweep.

Simple analogy:

Most tools read code like plain text. PreFlight reads code like a blueprint.

An AST, or Abstract Syntax Tree, is the structured shape of the code. Instead of just seeing words on a page, PreFlight sees what the code actually means: imports, function calls, variables, SQL strings, route handlers, and security-sensitive boundaries.

That matters because dangerous code can be written in many different ways. A basic text search might miss it. An AST-based scanner can understand structure.

Examples:

- ? It can tell when a variable named `apiKey` is assigned a hardcoded string.
- ? It can tell when a SQL query is being built with `+ userId`.
- ? It can tell when server-only logic is being pulled into a client component.

? It can tell when a Supabase table is created without RLS.

The technical phrase to say aloud:

Phase 1 is a zero-token, offline Tree-sitter AST pass that performs deterministic structural analysis before code leaves the developer's machine.

Translation:

It is fast, private, and does not need to call an AI model for basic safety checks.

Tree-sitter

Tree-sitter is the parsing engine PreFlight uses.

Simple analogy:

Tree-sitter is like an X-ray machine for code. It shows the skeleton underneath the text.

Why it is important:

- ? It can parse real code structure.
- ? It works locally.
- ? It is fast enough for CLI and watch-mode workflows.
- ? It is more reliable than only using regular expressions.

The pitch phrase:

PreFlight uses Tree-sitter to inspect the code's syntax tree, not just the raw text.

Tri-State Risk Score

PreFlight does not just dump warnings. It classifies risk into three categories.

Hard Block

Simple meaning:

Stop. This is dangerous enough that it should not ship.

Examples:

- ? Exposed secrets.
- ? Supabase service role leaks.
- ? Missing RLS.
- ? Raw SQL injection patterns.
- ? Obvious authorization bypasses.

Pitch phrase:

Hard Block means PreFlight found a confirmed production risk.

High-Risk Drift

Simple meaning:

The code may be dangerous, but it needs deeper context to prove.

Examples:

- ? Auth behavior changed across multiple files.
- ? Tenant isolation logic was rewritten.
- ? Webhook behavior changed without a clear idempotency guard.
- ? A route boundary looks suspicious but needs semantic review.

Pitch phrase:

High-Risk Drift means the AI changed an architectural boundary that should be reviewed before release.

Likely Safe

Simple meaning:

The change looks normal based on the structural rules PreFlight can verify.

Pitch phrase:

Likely Safe means the local structural scan did not detect a release-blocking risk.

Phase 2: MCP Patch Flow

Phase 2 is for the problems that are too complex for simple local rules.

Simple analogy:

Phase 1 is the airport metal detector. Phase 2 is the security officer who opens the bag and checks the suspicious item.

When PreFlight sees a complex architectural issue, it can route that issue to an MCP server that helps generate a patch.

MCP stands for Model Context Protocol. In simple terms, it lets developer tools talk to AI agents in a controlled, structured way.

Why it matters:

- ? It connects PreFlight findings to an auto-fix workflow.
- ? It gives the AI the relevant code context.
- ? It lets the user review the patch before accepting it.
- ? It turns "there is a problem" into "here is a safe proposed fix."

Pitch phrase:

Phase 2 uses MCP to convert high-risk structural findings into reviewable patches.

3. The Hackathon Upgrades

These are the four features being built around the existing PreFlight engine for the June 21 hackathon pitch.

Feature 1: The Eye, Watch Mode

What it does:

The Eye runs quietly in the background and watches the project while an AI coding agent writes files.

Instead of waiting for the developer to manually run a scan, PreFlight notices file changes instantly and triggers a safety check.

Simple analogy:

The Eye is a motion sensor for your codebase. The moment the AI touches a file, PreFlight looks at what changed.

Why this matters for vibe coding:

Vibe coding is fast. Developers may prompt an AI agent to build an entire app in minutes. That speed creates risk because dangerous changes can happen faster than a human can review them.

The Eye makes PreFlight feel alive. It watches the code as it changes, not after the damage is already buried.

Technical terms to say aloud:

- ? Watch Mode.
- ? Background daemon.
- ? File-system event watcher.
- ? Chokidar.
- ? Continuous local AST scanning.

Clean pitch sentence:

The Eye is a chokidar-powered background daemon that watches file-system changes and runs PreFlight the moment an AI agent writes code.

Demo moment:

1. Start The Eye in the terminal.
2. Ask an AI tool to generate risky code.
3. The terminal instantly lights up with a PreFlight warning.
4. The audience sees that PreFlight reacts in real time.

Why judges will understand it:

It turns PreFlight from "a scanner you run later" into "a guardrail that watches while AI codes."

Feature 2: Supabase Migration Parser

What it does:

This feature checks SQL migration files.

If an AI creates a new Supabase table, PreFlight verifies that the migration also includes Row-Level Security rules.

Simple analogy:

Creating a database table without RLS is like building an apartment building without locks on the doors.

Supabase is popular with AI-generated apps because it is fast to set up. But many new developers do not understand RLS deeply. AI tools may create tables and forget to enable proper policies.

PreFlight catches that.

Technical terms to say aloud:

- ? Supabase migration parser.
- ? SQL structural analysis.
- ? Row-Level Security enforcement.
- ? RLS policy verification.
- ? Tenant isolation.
- ? Database access control.

Clean pitch sentence:

The Supabase Migration Parser uses Tree-sitter to inspect SQL migrations and enforce that new tables include Row-Level Security before they ship.

Demo moment:

1. Show an AI-generated SQL migration that creates a table.
2. The migration looks normal.
3. PreFlight flags it because RLS is missing.
4. Then show the corrected version with `enable row level security` and policies.

Why judges will care:

This is not a theoretical lint rule. Missing RLS can expose user data in production.

Feature 3: 0-Token Fast Checkers

What they do:

These are ultra-fast local checks that cost nothing to run.

They do not call an AI model. They do not burn API tokens. They run instantly on the developer's machine.

The first checker is the Secrets Interceptor.

It catches leaked API keys, service keys, tokens, and credential-shaped strings before they get committed.

Simple analogy:

The Secrets Interceptor is like a smoke alarm for leaked credentials.

The second checker is the Supply-Chain Unpinner.

It checks `package.json` for unsafe dependency patterns like wildcard versions.

Simple analogy:

The Supply-Chain Unpinner is like checking whether your building materials came from a trusted supplier or a mystery box.

Why this matters:

AI agents often add dependencies quickly. They may use loose versions, wildcards, or unsafe package patterns. That can create supply-chain risk.

Technical terms to say aloud:

- ? 0-token checks.
- ? Local-only plugin system.
- ? Secrets interception.
- ? Dependency hygiene.
- ? Supply-chain risk.
- ? Package version pinning.
- ? Regex fast path.

Clean pitch sentence:

PreFlight combines AST-based structural checks with 0-token fast checkers for secrets and supply-chain hygiene, so basic risks are caught instantly without touching a cloud model.

Demo moment:

1. Paste a fake-looking secret into code.
2. PreFlight catches it immediately.
3. Add a wildcard dependency like `"some-package": "*" to package.json`.
4. PreFlight flags the supply-chain risk.

Why judges will care:

It shows PreFlight is practical and cost-aware. Not every safety check needs expensive AI reasoning.

Feature 4: TUI Release Gate

What it does:

The TUI Release Gate turns PreFlight output into a clean terminal dashboard.

Instead of messy logs, it gives the user a beautiful release screen showing:

- ? Current risk level.
- ? Files affected.
- ? Findings grouped by severity.
- ? What is safe.
- ? What is blocked.
- ? What can be patched.

The user can press **P** to send a finding into the MCP patch flow.

Simple analogy:

The TUI Release Gate is the cockpit dashboard before takeoff. It tells you whether the plane is cleared to fly.

Technical terms to say aloud:

- ? Terminal User Interface.
- ? Ink.
- ? Interactive release gate.
- ? Keyboard-driven patch workflow.
- ? Human-in-the-loop remediation.
- ? MCP-powered patch handoff.

Clean pitch sentence:

The TUI Release Gate is an Ink-powered terminal interface that turns PreFlight from a log stream into an interactive safety dashboard, with one-key MCP patch generation.

Demo moment:

1. Run PreFlight.

2. The terminal clears and renders a polished dashboard.
3. A risky finding appears as a Hard Block.
4. Press **P**.
5. PreFlight routes the finding to the MCP patch flow.
6. The proposed fix appears for review.

Why judges will care:

This makes the product feel real. The visual payoff is immediate, and the workflow is easy to understand even for non-technical people.

4. Non-Technical Glossary

AI Coding Drift

What it means:

AI coding drift is when an AI coding tool changes the structure or security behavior of your app in ways you did not intend.

Metaphor:

You asked someone to repaint the wall, but they quietly removed the front door lock.

How to say it:

PreFlight catches AI Coding Drift before it reaches production.

AST

Full term:

Abstract Syntax Tree.

What it means:

An AST is the structured blueprint of code.

Metaphor:

If code text is the book, the AST is the table of contents, grammar, and sentence structure all at once.

How to say it:

We inspect the AST, so we can reason about code structure instead of searching plain text.

Tree-sitter

What it means:

Tree-sitter is the parsing engine that turns code into an AST.

Metaphor:

Tree-sitter is the X-ray scanner. It lets PreFlight see the skeleton of the code.

How to say it:

We use Tree-sitter for fast, local, deterministic parsing.

Regex

Full term:

Regular expression.

What it means:

Regex is a pattern-matching tool for text.

Metaphor:

Regex is like a sniffer dog trained to recognize specific smells, such as API keys.

How to say it:

We use Regex only for fast, obvious patterns like credential-shaped strings, not for deep architecture analysis.

MCP

Full term:

Model Context Protocol.

What it means:

MCP is a standard way for tools to give context to AI agents and receive structured actions back.

Metaphor:

MCP is the translator between PreFlight and the AI repair crew.

How to say it:

MCP lets PreFlight turn a risky finding into a reviewable patch workflow.

Daemon

What it means:

A daemon is a background process that keeps running while you work.

Metaphor:

A daemon is like a security guard who stays on duty even when you are not actively looking.

How to say it:

The Eye runs as a background daemon during development.

Chokidar

What it means:

Chokidar is a file-watching library for Node.js.

Metaphor:

Chokidar is the motion sensor that notices when a file changes.

How to say it:

Watch Mode uses chokidar to detect AI-written file changes in real time.

RLS

Full term:

Row-Level Security.

What it means:

RLS is a database security system that controls which rows each user is allowed to access.

Metaphor:

RLS is the lock on every apartment door in your database.

How to say it:

PreFlight checks whether AI-generated Supabase migrations include proper RLS protections.

Supply Chain

What it means:

In software, the supply chain is the set of third-party packages your app depends on.

Metaphor:

Your app is a building, and packages are the materials. If the materials are unsafe, the building is unsafe.

How to say it:

PreFlight checks dependency hygiene so AI agents do not quietly introduce supply-chain risk.

TUI

Full term:

Terminal User Interface.

What it means:

A TUI is an interactive app that runs inside the terminal.

Metaphor:

It is a dashboard inside the command line.

How to say it:

The Release Gate is an Ink-powered TUI that gives developers a cockpit view before they ship.

Ink

What it means:

Ink is a library for building React-style terminal interfaces.

Metaphor:

Ink lets us paint a real app inside the terminal.

How to say it:

We use Ink to turn PreFlight's scan results into a polished interactive release gate.

Pitch Cheat Sheet

One-Liner

PreFlight is a local-first safety gate that catches AI Coding Drift before it becomes production technical debt.

Technical One-Liner

PreFlight combines Tree-sitter AST analysis, 0-token local security checkers, Tri-State risk scoring, and MCP-powered remediation into a release gate for AI-generated code.

Problem Statement

AI coding agents make developers faster, but they also create invisible architectural and security drift. PreFlight gives teams a local guardrail between AI-generated code and production.

Why Now

The more code AI writes, the less realistic it is for humans to manually review every auth boundary, database policy, route handler, and dependency change. PreFlight automates the first safety pass.

What Makes It Different

PreFlight is not a generic scanner. It is designed around the exact failure modes of vibe-coded apps: leaked service keys, broken Supabase RLS, raw SQL, middleware drift, webhook idempotency, and AI-generated architecture changes.

Best Demo Story

1. An AI agent writes a risky Supabase or Next.js change.
2. The Eye detects the file change instantly.
3. PreFlight runs a local Tree-sitter AST sweep.
4. The TUI Release Gate shows a Hard Block.
5. The founder presses P.
6. MCP generates a reviewable patch.
7. The app moves from unsafe to ready-to-review.

Founder Confidence Line

We are not trying to replace developers. We are building the missing safety layer between AI speed and production trust.